In-Class Exercise: Recursive Binary Search

Prof. Jordan C. Hanson

October 21, 2026

1 The Idea of Binary Search

Binary search finds a target in a **sorted** list by repeatedly checking the midpoint and discarding the half that cannot contain the target. Use the list

```
values = [3, 7, 11, 18, 23,
          29, 34, 42, 47, 53,
          61, 68, 74, 83, 91]
```

The indices run from 0 through 14. Suppose the target is 74. Initially,

$$\texttt{low} = 0, \qquad \texttt{high} = 14.$$

Compute the midpoint index with

$$\texttt{mid} = \left\lfloor \frac{\texttt{low} + \texttt{high}}{2} \right\rfloor.$$

The oddly shaped brackets indicate the *floor function*, or "rounding down." Complete the trace:

low	high	mid	value	keep
0	14	___	____	_____
___	___	___	____	_____
___	___	___	____	_____
___	___	___	____	_____

What index should the search return? _____

2 Base Case and Recursive Cases

A recursive binary search keeps the original list and changes only the indices `low` and `high`. The **base case** for an unsuccessful search occurs when `low` > `high`. There are then no indices left to search, so return `-1`. If `values[mid]` equals the target, return `mid`. Otherwise:

- if the target is smaller, search `low` through `mid-1`;
- if the target is larger, search `mid+1` through `high`.

3 Complete the Recursive Function

Fill in the missing expressions.

```
def binary_search(values,target,low,high):
    if ____________________:
        return -1

    mid = ___________________

    if values[mid] == target:
        return ______________

    if target < values[mid]:
        return binary_search(values,target,low,____)

    return binary_search(values,target,____,high)
```

Call the function using the entire list:

```
result = binary_search(values,74,0,len(values)-1)
print(result)
```

Predict the output before running it: _____

4 Search for a Missing Value

Now search for 50:

```
result = binary_search(values,50,0,len(values)-1)
```

Trace the sequence of midpoint values examined. What value should the function return? Explain why `low > high` eventually becomes true:

5 Why Binary Search Is Efficient

Each recursive call discards about half of the remaining search region. A list of roughly $1024 = 2^{10}$ sorted values therefore needs at most about 10 midpoint decisions, rather than checking every value one at a time.

Question: Why must the list be sorted for this method to work?